

Real ABA PayWay & Bakong KHQR Integration Guide

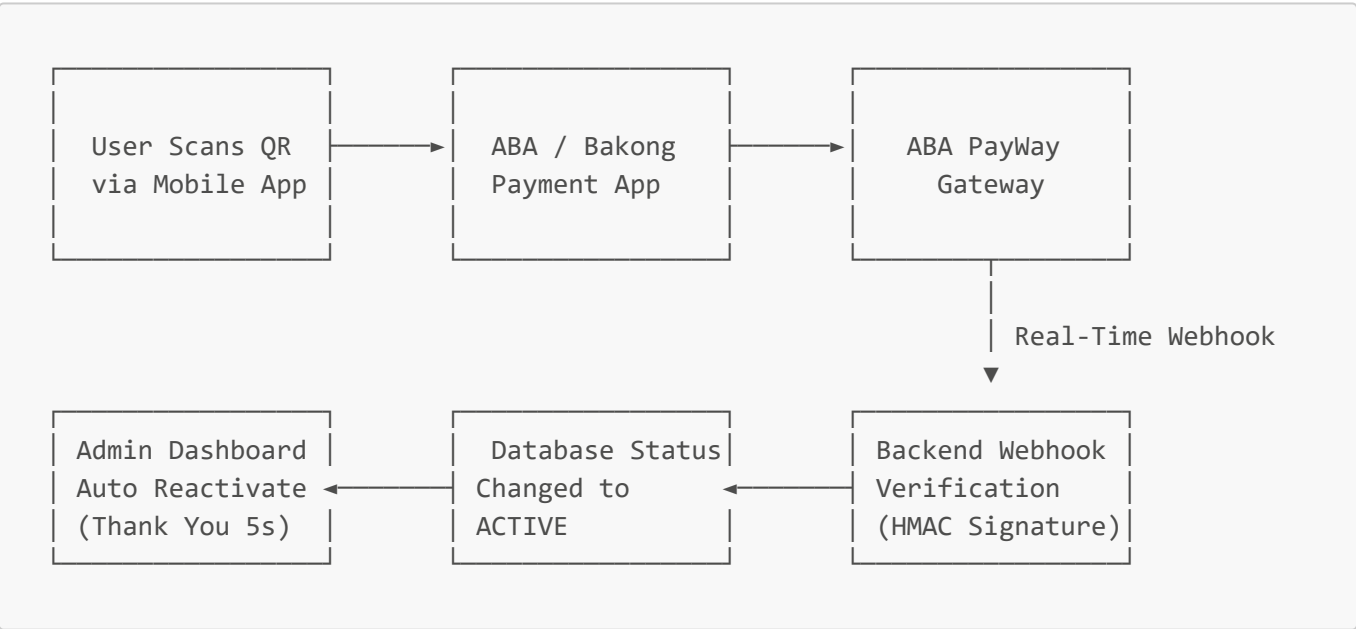
This document provides a complete, step-by-step technical guide to integrating **real ABA PayWay & Bakong KHQR automatic payment verification** into the attendance management system.

Table of Contents

- 1. [Overview & Architecture](#)
- 2. [Step 1: Obtain ABA PayWay Merchant Credentials](#)
- 3. [Step 2: Configure Environment Variables](#)
- 4. [Step 3: Backend Dynamic KHQR Code Generation](#)
- 5. [Step 4: Secure Webhook Callback Implementation](#)
- 6. [Step 5: Register Webhook URL in ABA Merchant Portal](#)
- 7. [Step 6: Frontend Auto-Detection & Thank You Screen](#)
- 8. [Testing & Go-Live Checklist](#)

1. Overview & Architecture

When an admin organization is **SUSPENDED**, the system displays an official **Bakong KHQR payment card** with the designated amount (e.g., **\$0.01 USD**).



Step 1: Obtain ABA PayWay Merchant Credentials

To accept real payments via ABA Mobile and Bakong, you must register as an ABA Merchant:

- 1. **Apply for ABA PayWay Account:**
 - Contact **ABA Bank Merchant Services** or register at payway.com.kh.

- Obtain your **Merchant ID** (e.g. `eroxii_vireak`).
- Obtain your secret **API Key** from the ABA PayWay Merchant Portal.

2. Environment URLs:

- **Sandbox URL:** `https://checkout-sandbox.payway.com.kh/api/v1/`
- **Production URL:** `https://checkout.payway.com.kh/api/v1/`

Step 2: Configure Environment Variables

Add the ABA credentials to your backend configuration file `backend/.env` and `.env.docker`:

```
# =====  
# ABA PAYWAY & BAKONG KHQR CONFIGURATION  
# =====  
ABA_PAYWAY_MERCHANT_ID="eroxii_vireak"  
ABA_PAYWAY_API_KEY="YOUR_ABA_PAYWAY_SECRET_API_KEY"  
ABA_PAYWAY_API_URL="https://checkout.payway.com.kh/api/v1/"  
ABA_BAKONG_ACCOUNT_ID="eroxii_vireak@aba"  
  
# Public Webhook Domain (Cloudflare Tunnel URL)  
FRONTEND_URL="https://beatles-epic-however-hair.trycloudflare.com"
```

Step 3: Backend Dynamic KHQR Code Generation

Install the official Bakong KHQR npm library in `backend`:

```
cd backend  
npm install bakong-khqr
```

NestJS Implementation Example (`backend/src/admin/admin.service.ts`)

```
import { Injectable } from '@nestjs/common';  
import { BakongKHQR, khqrData, IndividualInfo } from 'bakong-khqr';  
  
@Injectable()  
export class AdminService {  
  // Generate a dynamic KHQR string for standard $0.01 payment  
  generateOrgPaymentQr(orgId: number, companyName: string, amount: number = 0.01)  
  {  
    const optionalData = {  
      currency: khqrData.currency.usd,  
      amount: amount,  
      mobileNumber: '85512345678',  
      storeLabel: companyName.substring(0, 25),  
      terminalLabel: `ORG-${orgId}`,  
    };  
  }  
}
```

```

        billNumber: `INV-${orgId}-${Date.now()}`,
    };

    const individualInfo = new IndividualInfo(
        process.env.ABA_BAKONG_ACCOUNT_ID || 'eroxii_vireak@aba',
        companyName,
        'Phnom Penh',
        optionalData,
    );

    const khqr = new BakongKHQR();
    const response = khqr.generateIndividual(individualInfo);

    if (response.status.code === 0) {
        return {
            qrString: response.data.qr,
            md5: response.data.md5,
            billNumber: optionalData.billNumber,
        };
    } else {
        throw new Error(`KHQR Generation Failed: ${response.status.message}`);
    }
}

```

Step 4: Secure Webhook Callback Implementation

When a customer scans and completes payment, ABA PayWay posts a JSON payload to your backend. The endpoint **must verify the HMAC SHA-512 signature** before marking the organization **ACTIVE**.

Controller Endpoint ([backend/src/admin/admin.controller.ts](#))

```

import { Controller, Post, Body, HttpStatusCode, HttpStatus, Logger } from
'@nestjs/common';
import * as crypto from 'crypto';
import { AdminService } from './admin.service';

@Controller('admin')
export class AdminController {
    private readonly logger = new Logger(AdminController.name);

    constructor(private readonly adminService: AdminService) {}

    /**
     * Real ABA PayWay Webhook Receiver
     */
    @Post('payment-webhook')
    @HttpCode(HttpStatus.OK)
    async handleAbaPaymentWebhook(@Body() payload: any) {
        this.logger.log(`Received ABA Webhook: ${JSON.stringify(payload)}`);
    }
}

```

```

const merchantId = payload.merchant_id;
const tranId = payload.tran_id;
const status = payload.status; // 0 indicates success
const amount = payload.amount;
const receivedHash = payload.hash;

// 1. Verify HMAC SHA-512 Signature
const apiKey = process.env.ABA_PAYWAY_API_KEY;
const rawData = `${tranId}${merchantId}${amount}${status}`;
const calculatedHash = crypto
  .createHmac('sha512', apiKey)
  .update(rawData)
  .digest('hex');

if (calculatedHash.toLowerCase() !== receivedHash?.toLowerCase()) {
  this.logger.error('Invalid ABA PayWay Webhook Signature!');
  return { status: 'ERROR', message: 'Invalid hash signature' };
}

// 2. Process Successful Payment (status === 0 or '0')
if (String(status) === '0') {
  // Extract Organization ID from transaction reference/bill number
  // Format example: billNumber "INV-12-1725523200" -> orgId 12
  const billNumber = payload.bill_number || '';
  const orgIdMatch = billNumber.match(/INV-(\d+)-/);
  const orgId = orgIdMatch ? parseInt(orgIdMatch[1], 10) : null;

  if (orgId) {
    await this.adminService.updateOrgStatus(orgId, 'ACTIVE');
    this.logger.log(`Successfully activated Organization ID: ${orgId}`);

    // Optional: Send Telegram notification to Admin
    await this.adminService.notifyAdminPaymentSuccess(orgId, amount, tranId);
  }
}

return { status: 'OK', message: 'Webhook processed successfully' };
}

```

Step 5: Register Webhook URL in ABA Merchant Portal

1. Log into your **ABA PayWay Merchant Portal** (payway.com.kh).
2. Navigate to **Developer Settings** -> **Webhook Configuration**.
3. Set **Payment Push Notification URL** to:

```
https://beatles-epic-further-hair.trycloudflare.com/api/admin/payment-webhook
```

(Replace with your active Cloudflare Tunnel domain or custom domain).

4. Enable HTTP **POST** notifications for **SUCCESSFUL_TRANSACTION**.

Step 6: Frontend Auto-Detection & Thank You Screen

The frontend **AdminLayout.vue** automatically polls the organization status every 3 seconds while suspended:

```
// Background polling loop in AdminLayout.vue
onMounted(() => {
  if (isCurrentOrgSuspended.value) {
    statusPollingTimer = window.setInterval(async () => {
      try {
        const res = await adminApi.getSettings();
        const latestStatus = res.data?.status;

        if (latestStatus === 'ACTIVE') {
          clearInterval(statusPollingTimer);

          // Trigger 5-Second Thank You Overlay
          showThankYouScreen.value = true;
          startThankYouCountdown();

          // Sync Pinia store after countdown
          setTimeout(() => {
            adminStore.setSettings(res.data);
            showThankYouScreen.value = false;
          }, 5000);
        }
      } catch (err) {
        console.error('Polling check failed', err);
      }
    }, 3000);
  }
});
```

Testing & Go-Live Checklist

- ☐ **Step 1:** Register ABA PayWay Merchant account and obtain **ABA_PAYWAY_MERCHANT_ID** & **ABA_PAYWAY_API_KEY**.
- ☐ **Step 2:** Add credentials to **backend/.env** and **.env.docker**.
- ☐ **Step 3:** Deploy public HTTPS Webhook URL via Cloudflare Tunnel.
- ☐ **Step 4:** Test payment scan in ABA PayWay Sandbox with test mobile app.
- ☐ **Step 5:** Verify Webhook signature in NestJS logs (**Received ABA Webhook**).
- ☐ **Step 6:** Confirm DB status updates to **ACTIVE** and 5-second Thank You popup displays cleanly.
- ☐ **Step 7:** Switch **ABA_PAYWAY_API_URL** to Production for live operations.